COMPLETE BEGINNER'S GUIDE — UPDATED

Recruiter AI

How this project was built, and how every part of the code works — explained from zero.

Repo: **github.com/Harbringer904/agent_max** · Stack: Node.js + Express + vanilla JS

What this guide assumes: nothing. If you have never written code, you can still read this top-to-bottom and understand the whole project. Terms are defined the first time they appear, and again in the Glossary at the end.

How to read it

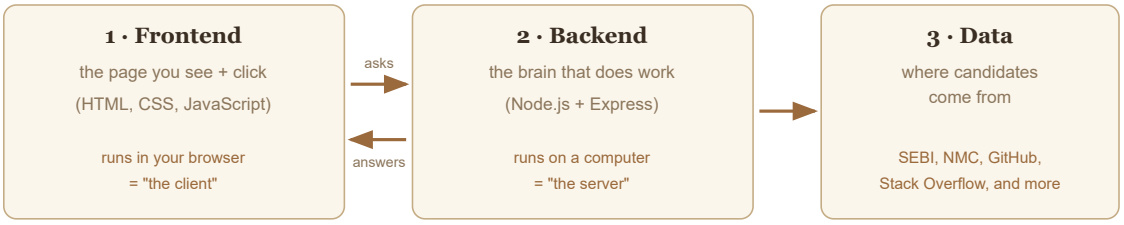
- Every section is short bullets + a diagram, not walls of text.
- Boxes like the one above are "plain-English" notes.
- Grey code blocks show the *actual* code or data from the project.
- The **Glossary** (last page) defines every technical word in one line.

1. What we built (in one breath)

- A **website** where a recruiter types what they want in a candidate (skills, years of experience, certifications, etc.), and the app returns a **ranked list of people**, each with a **score out of 100** and a **rating out of 10**.
- It can pull candidates from **8 different sources** — including two **real official government registries** — and works for **any job field**, not just developers.

2. Web-app foundations (skip if you know these)

- A **website** is made of three parts that talk to each other:



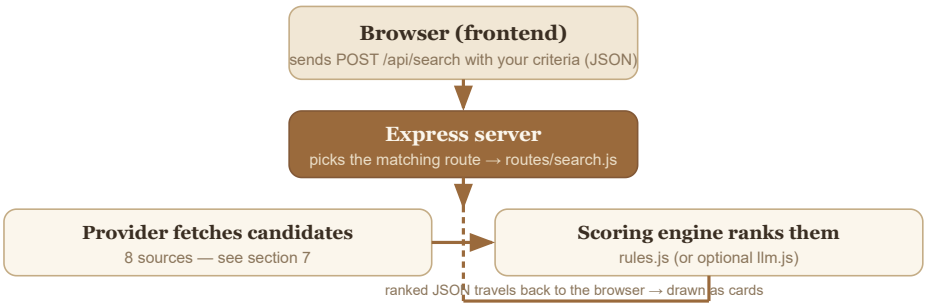
- **Client** = your browser (the thing showing the page). **Server** = a program running elsewhere that the client talks to.
- **API** = the "menu" of requests the server accepts. The client orders from the menu; the server cooks and returns a result. (Our menu items start with `/api/…` .)
- **JSON** = the plain-text format used to pass data between client and server. It looks like `{ "name": "Ada", "skills": ["python"] }` .

3. The tech stack — what each tool is, and why we used it

Tool	What it is (one line)	Why it's here
JavaScript	The programming language of the web.	The whole app is written in it — front and back.
Node.js	Lets JavaScript run *outside* a browser (on a server).	Powers the backend.
Express	A small Node library for building web servers + APIs.	Handles incoming requests and routes them.
HTML / CSS	Page structure / page looks.	The user interface, in the warm "woody" theme.
JSON	A text format for structured data.	How data moves client ↔ server.
npm	Node's "app store" for code libraries + a task runner.	Installs Express/dotenv; runs `npm test`.
REST API	A style of API using URLs + verbs (GET/POST).	Our `/api/...` endpoints.
Git / GitHub	Change tracking / project hosting.	Version control + publishing.
LLM	"Large Language Model" — an AI like Claude/Groq/Gemini.	*Optional* human-readable reasoning for each candidate.
`.env` file	A private file holding secrets (API keys/tokens).	Keeps keys out of the code and out of GitHub.

> **Key fact:** the app still uses only **two** installed libraries — `express` and `dotenv`. Every new data source (SEBI, NMC, OpenStreetMap, Google Places) was built by hand with plain HTTP calls, no extra dependencies.

4. The big picture — how a request flows



- The **folder structure** mirrors those jobs — each folder has one responsibility:

```
recruiter-ai/
├── server.js           ← starts the server, connects the routes
```

```
├── routes/               ← health / fields / templates / search / results
├── lib/
│   ├── normalize.js       ← forces every candidate into one common shape
│   ├── jobTemplates.js    ← 14 pre-built job templates
│   ├── resultsStore.js    ← saved/shareable result sets
│   ├── scoring/
│   │   ├── rules.js       ← the math that turns a match into a 0–100 score
│   │   └── llm.js         ← optional AI scoring + reasoning (Groq/Gemini/Claude)
│   └── providers/         ← the 8 data sources, one file each
│       ├── index.js       ├── github.js       ├── stackoverflow.js
│       ├── sample.js      ├── upload.js       ├── googlePlaces.js
│       ├── osm.js         ├── sebiRia.js      └── nmc.js
├── data/samples/*.json     ← demo candidate pools per field
├── data/registries/         ← real SEBI data snapshot
├── scripts/fetch-sebi-ria.mjs  ← re-downloads the SEBI snapshot
├── public/index.html       ← the entire frontend (one file)
└── test/*.test.js          ← 68 automated checks
```

5. The core idea — turn "requirements" into a score

Same as before — 4 data shapes: **Candidate**, **JobSpec**, **Criterion**, **Scored candidate**. See the worked example below (still applies to every source, real or demo).

Candidate — one person/professional, in a standard form (every source is converted to this):

```
{ "name": "Surachit Kumar", "field": "healthcare",
  "yearsExperience": 22, "education": "MBBS",
  "certifications": ["NMC Registered"],
  "location": "Ghaziabad", "source": "nmc" }
```

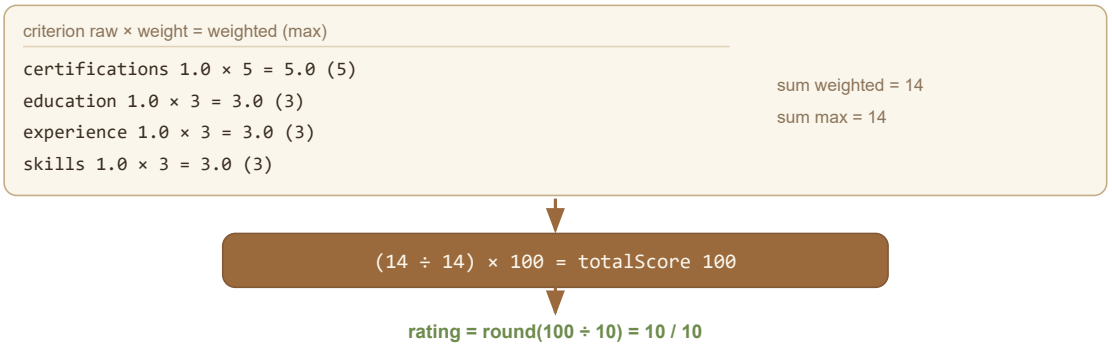
6 criterion types, each producing a raw match between **0** and **1**:

key	What it measures	How raw (0–1) is decided
`skills`	Required skills present	(matched skills) ÷ (required skills)
`experience`	Years of experience	candidate years ÷ required years (capped at 1)
`education`	Highest degree	meets required level → 1, else proportional
`certifications`	Required certificates held	(matched certs) ÷ (required certs)
`location`	Location match	matches → 1, else 0
`keyword`	Words in their summary/skills	(matched keywords) ÷ (total keywords)

6. The scoring engine — the math, made simple

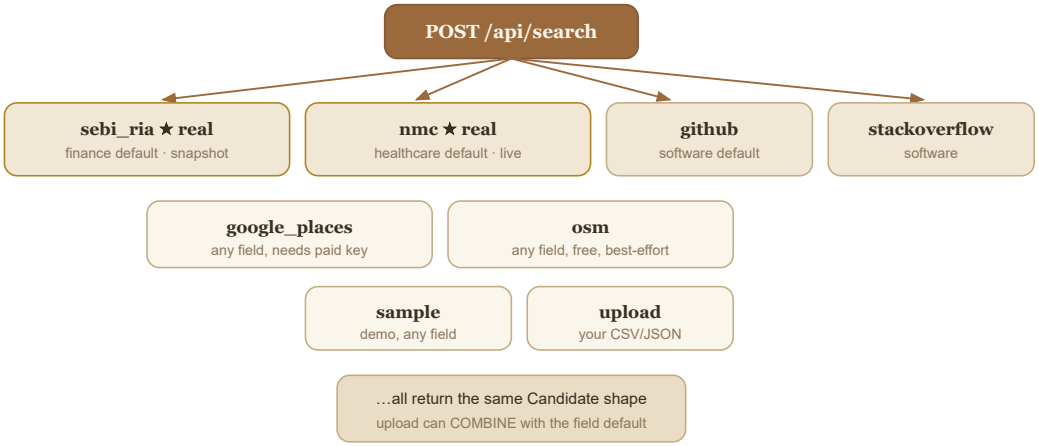
`weighted = raw × weight` . `totalScore = (Σ weighted ÷ Σ max) × 100` . `rating = round(totalScore ÷ 10)` .

Worked example — a nurse vs. the "Registered Nurse" job



- **A "Required" checkbox** on any criterion turns it into a hard filter: candidates who don't fully meet it (raw < 1) are dropped from the results entirely, not just scored lower.
- Missing data never crashes it — a criterion with no data scores 0 with a note like `"no education data"` .

7. Where candidates come from — now 8 sources, 2 of them official government data



Provider	Source	Real?	Notes
sebi_ria	Official SEBI registry (India)	✅ Real	1,042 financial advisers, bundled snapshot, **finance default**
nmc	Official NMC registry (India)	✅ Real	31,000+ doctors under Delhi alone, **live query**, **healthcare default**
github	GitHub's public API	✅ Real	developers, **software default**
stackoverflow	Stack Exchange API	✅ Real	developers, free, no key
google_places	Google Places (New)	✅ Real	any field/city, needs a **paid-tier** key (free credit, card required)
osm	OpenStreetMap + Overpass	✅ Real	any field/city, **free, no key**, but coverage/uptime is best-effort
sample	Bundled JSON	❌ Fake	demo data, any field
upload	Your CSV/JSON	✅ Real (yours)	can be **combined** with the field's default source

8. Two new REAL government data sources, explained

SEBI (finance) — a downloaded snapshot

- SEBI (India's stock market regulator) publishes a **public, searchable list of Registered Investment Advisers** — meant for investors to verify an adviser before trusting them with money.
- We wrote a one-off script (`scripts/fetch-sebi-ria.mjs`) that downloads the whole list (1,042 real advisers — name, registration number, address, since-when-registered) and saves it as a file the app reads instantly. It does **not** call SEBI on every search — that would be disrespectful of a public service.
- Delhi alone: **55 real, named financial advisers**.

NMC (healthcare) — a live query

- NMC (India's medical regulator) publishes a **public doctor registry** — same idea as SEBI, but far too big to download as one file (**1,000,000+ doctors nationally**; 31,588 under Delhi's council alone).
- So instead of a snapshot, the app calls NMC's own API **live**, every search, asking only for the state you searched (India has no city-level doctor registry — "Delhi" maps to "Delhi Medical Council").
- For each doctor found, it makes one more quick call to get their real degree, university, and address.

> **A small technical wrinkle, explained simply:** when a browser or server connects to a website securely (https), the website proves who it is with a "certificate" — like an ID card, backed by a trusted authority. NMC's website has a real ID card, but its server forgets to show the ID card's "backing letter" (called an intermediate certificate). Web browsers know how to fetch that backing letter themselves, so people never notice. Strict tools like our server's normal connection code do not, and refuse to connect. We turned off that one strict check, **only for NMC's own requests** — every other connection in the app (GitHub, SEBI, the AI providers) still does the full, normal, strict check. This is documented in the code, README, and FAIRNESS.md so nothing is hidden.

9. How we found these sources — real detective work, not guessing

- Both SEBI and NMC's data pages are built the old-fashioned way: a webpage with a search button that quietly asks the server for data behind the scenes (this is called **AJAX**).
- We read the website's own JavaScript file (the same code your browser downloads and runs) to find the *exact* web address and parameters that button click sends — then replicated that exact request ourselves with `curl` / `fetch` , instead of guessing.
- We tried this same approach for **ICAI** (India's Chartered Accountants body) and it did **not** work cleanly — their member data is scattered across dozens of separate PDF files with no single searchable list. Rather than build something fragile and pretend it works, **we left it out** and said so honestly in `FAIRNESS.md` .
- Same story for **Google Places**: it's real and reliable, but Google requires a credit card on file to issue a key — so it's documented as optional, not the default anywhere.
- Same story for **OpenStreetMap**: it's genuinely free, but during testing the free public servers gave inconsistent results (sometimes fast, sometimes "too many requests"). We shipped it anyway — but honestly labeled it "best-effort," and it is never picked as an automatic default.

10. The "combine my CSV with your database" feature

- On the **Upload** data source, there's now a checkbox: *"Also include results from our database for this field."*
- Unchecked → searches **only** the CSV/JSON you pasted in.
- Checked → your candidates are merged with that field's default real source (SEBI for finance, NMC for healthcare, sample elsewhere) **before** scoring, so you see both side by side, ranked together.

```
recruiter uploads 1 candidate  +  provider="upload", combineWithDefault:true
        ↓
   server fetches your 1 candidate  +  25 from SEBI/NMC/sample
        ↓
   merges (removes exact duplicates)  →  scores everyone together  →  ranked list
```

11. Backend code, file by file (updated)

- **`routes/search.js`** — now also handles the "Required" hard filter and the CSV-combine merge, on top of the original search logic.
- **`routes/results.js`** — saves/reopens shareable result links (unchanged from before).
- **`lib/providers/sebiRia.js`** — reads the bundled SEBI snapshot file, filters by location.
- **`lib/providers/nmc.js`** — makes live calls to NMC, maps cities to State Medical Councils, enriches each doctor with real details.
- **`lib/providers/googlePlaces.js`** / **`osm.js`** — real-business search for any field, one calls Google (paid-tier key), the other calls free OpenStreetMap services.
- Everything else (`normalize.js` , `jobTemplates.js` , `scoring/rules.js` , `scoring/llm.js`) works exactly as described in the original guide — untouched contracts, just more data flowing through them.

12. The frontend — now shows every real source

- The **Data Source** panel used to only show 3 hardcoded choices. It's now **built dynamically** from the server's own list of registered providers — so every real source (including ones added after launch) automatically appears and is selectable, with the best default pre-picked for whichever field you chose.
- The **Required** checkbox next to each criterion, and the **combine-with-database** checkbox in the Upload panel, are the two new controls.

13. How this was actually built — the agentic method, continued

- Same method as before: **Opus 4.8 plans**, **Sonnet 5 agents implement**, a **verify agent** checks the work before anything ships.
- New work since the last guide, done the same way:

What	How it was found/built	Agents
Warm "woody" redesign	Sonnet-5 CSS reskin, JS untouched, verified	2
Google Places provider	Built against Google's documented API	—
SEBI real data + CSV-combine	Live reconnaissance (curl, reading site JS) + build + verify	—
Free OpenStreetMap provider	Live-tested for reliability before shipping	—
NMC real doctor registry	Live reconnaissance (site JS, endpoint testing, TLS diagnosis) + build	—

- Total across the whole project so far: **6 workflows, 20+ agents, 68 automated tests, all green.**

14. Run it yourself — step by step (unchanged)

1. **Install Node.js** (v18+) from nodejs.org.
2. `cd recruiter-ai`
3. `npm install`
4. `node server.js`
5. Open **http://localhost:3000** — SEBI and NMC already work with zero setup.
6. *(Optional)* Free AI reasoning: get a free key at **console.groq.com**, add `GROQ_API_KEY=...` to a new `.env` file, restart, tick 🧠 **AI reasoning**.
7. *(Optional)* Free OpenStreetMap search: nothing to configure — just pick it as a data source.
8. *(Optional, needs a card)* Google Places: get a key at console.cloud.google.com, add `GOOGLE_PLACES_API_KEY=...` .
9. `npm test` → expect `68 passing` .

15. Glossary (unchanged terms + a few new ones)

- **API, Backend, Frontend, Node.js, Express, REST, GET/POST, JSON, CSV, Provider, Criterion, Weight, Raw score, totalScore, Deterministic, LLM, Fallback, localStorage, Environment variable, Git, GitHub, Commit,** `.gitignore` **, Dependency, npm, Agent/workflow, Path-traversal** — see the original definitions; all still apply exactly as before.
- **Snapshot** — a saved copy of data downloaded once, used instead of asking the original website every time (what `sebi_ria` does).
- **Live query** — asking the original source fresh, every single search (what `nmc` , `github` , `osm` do).
- **AJAX** — a webpage quietly asking its own server for data in the background, without reloading the whole page.
- **TLS / certificate / intermediate certificate** — the "ID card" system that proves a website is really who it claims to be over a secure (https) connection; an intermediate certificate is a supporting document in that chain of trust.
- **Rate limit** — a service saying "you've asked too many times, slow down" (usually with an error like `429`).

- **Regulatory registry** — an official government list of licensed professionals (doctors, financial advisers) that the public can check.

— End of guide —

Recruiter AI · github.com/Harbringer904/agent_max